

Problem sets

Ejercicio 1:

Parte 1

preorden -XBAJRTMSWCZF

inorden -ARJTBXMSCWFZ

POSTORDEN-RTJABCFZWSMX

1)

3- tiene cuatro nodos externos, que son los bajos

2)

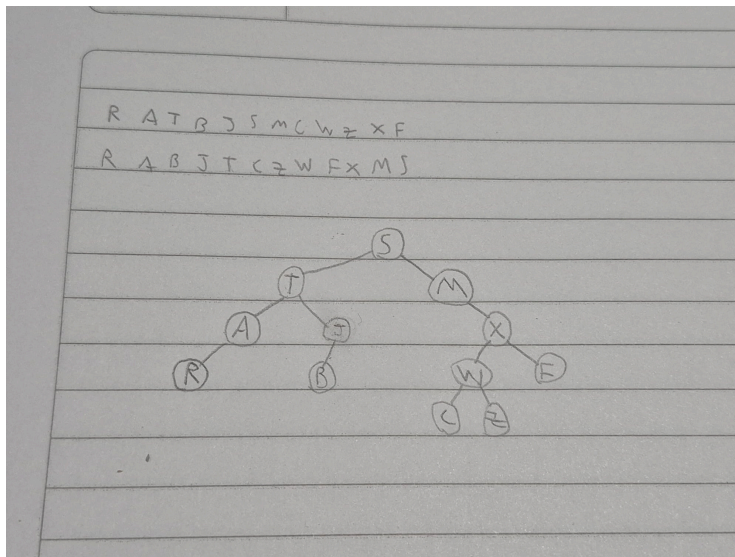
2- el inorden es correcto

3)

2- R aparece antes que A porque es descendiente de ese nodo

3- A aparece antes que T porque es ancestro de ese nodo

Parte 2



1)

4- Ninguna es correcta

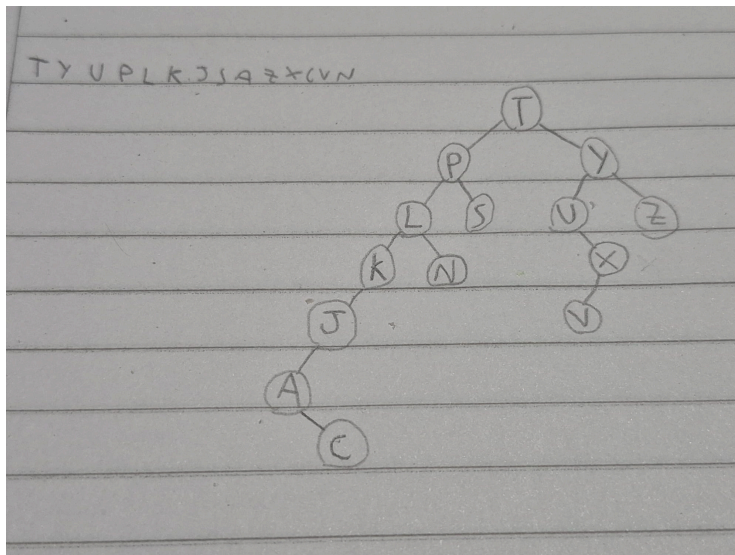
2)

4- Todas son correctas

3)

1- El árbol tiene altura 4

Parte 3



- 1)
4- "S" es una hoja

- 2)
1- El árbol tiene cinco hojas

Parte 4

- a) 1- calcula la altura del árbol
b) 1- $O(n)$ porque tienes que recorrer todo el árbol

Parte 5

Opción correcta 1. G es descendiente de C

Parte 6

- 1)
Preorden: 12, 1, 2, 7, 5, 25, 14, 33, 88, 45, 66, 99
Inorden: 1, 2, 5, 7, 12, 14, 25, 33, 45, 66, 88, 99
Postorden: 5, 7, 2, 1, 14, 66, 45, 99, 88, 33, 25, 12

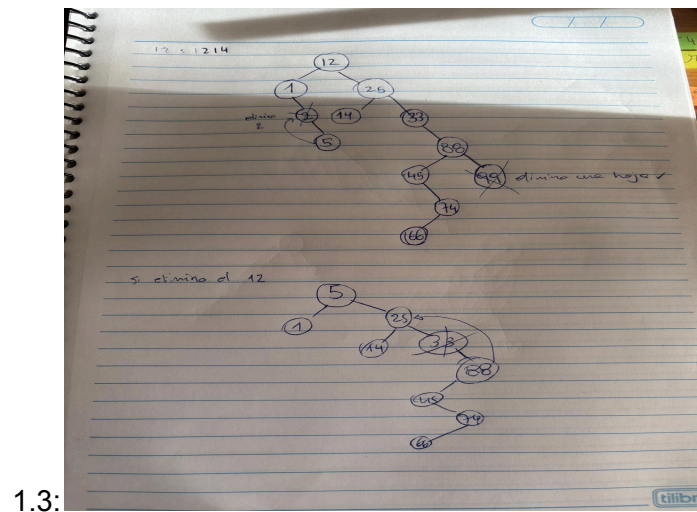
- 2)
a)
b) Opción 4) 12, 1, 2, 7, 5, 25, 14, 33, 88, 45, 66, 99
c) Opción 2) 1, 2, 5, 7, 12, 14, 25, 33, 45, 66, 88, 99

Parte 7

	$i(n) < i(m)$	$s(n) < s(m)$	$p(n) < p(m)$
n es descendiente de m	V	V	X
n esta a la izquierda de m	V	V	V
n esta a la derecha de m	X	X	X
n es ancestro de m	V	X	V

Parte 8

Ejercicio 3:



Ejercicio 4:

Sub-Equipo A

Descripción en lenguaje natural:

La altura de un árbol es la cantidad de niveles que tiene.

Un árbol vacío tiene altura 0, a partir de ahí, cada nivel que se agrega suma 1.

Se calcula mirando el hijo izquierdo y el derecho por separado, quedándose con el que sea más alto de los dos.

Pre-condición: el árbol puede estar vacío o tener cualquier cantidad de nodos.

Post-condición: se devuelve un entero ≥ 0 . El árbol queda exactamente igual que antes.

Casos de prueba:

1. Árbol vacío $\rightarrow$ altura 0.
2. Árbol de un solo nodo $\rightarrow$ altura 1.
3. Árbol bien balanceado con 7 nodos $\rightarrow$ altura 3.

Pseudocódigo:

```
function altura(nodo)
  si nodo es null entonces
    retornar 0
  fin si
  alturalzq <- altura(nodo.hijolzquierdo)
  alturaDer <- altura(nodo.hijoDerecho)
  retornar 1 + maximo(alturalzq, alturaDer)
fin function
```

Complejidad: $O(n)$ - porque recorre todos los nodos del árbol una sola vez

Sub-Equipo B

Descripción en lenguaje natural:

El tamaño de un árbol es cuántos nodos tiene en total. Se calcula sumando 1 (por el nodo actual) más el tamaño del subárbol izquierdo más el tamaño del subárbol derecho.

Pre-condición: el árbol puede estar vacío o tener cualquier cantidad de nodos.

Post-condición: se devuelve un entero ≥ 0 que es la cantidad de nodos.

El árbol no se modifica.

Casos de prueba:

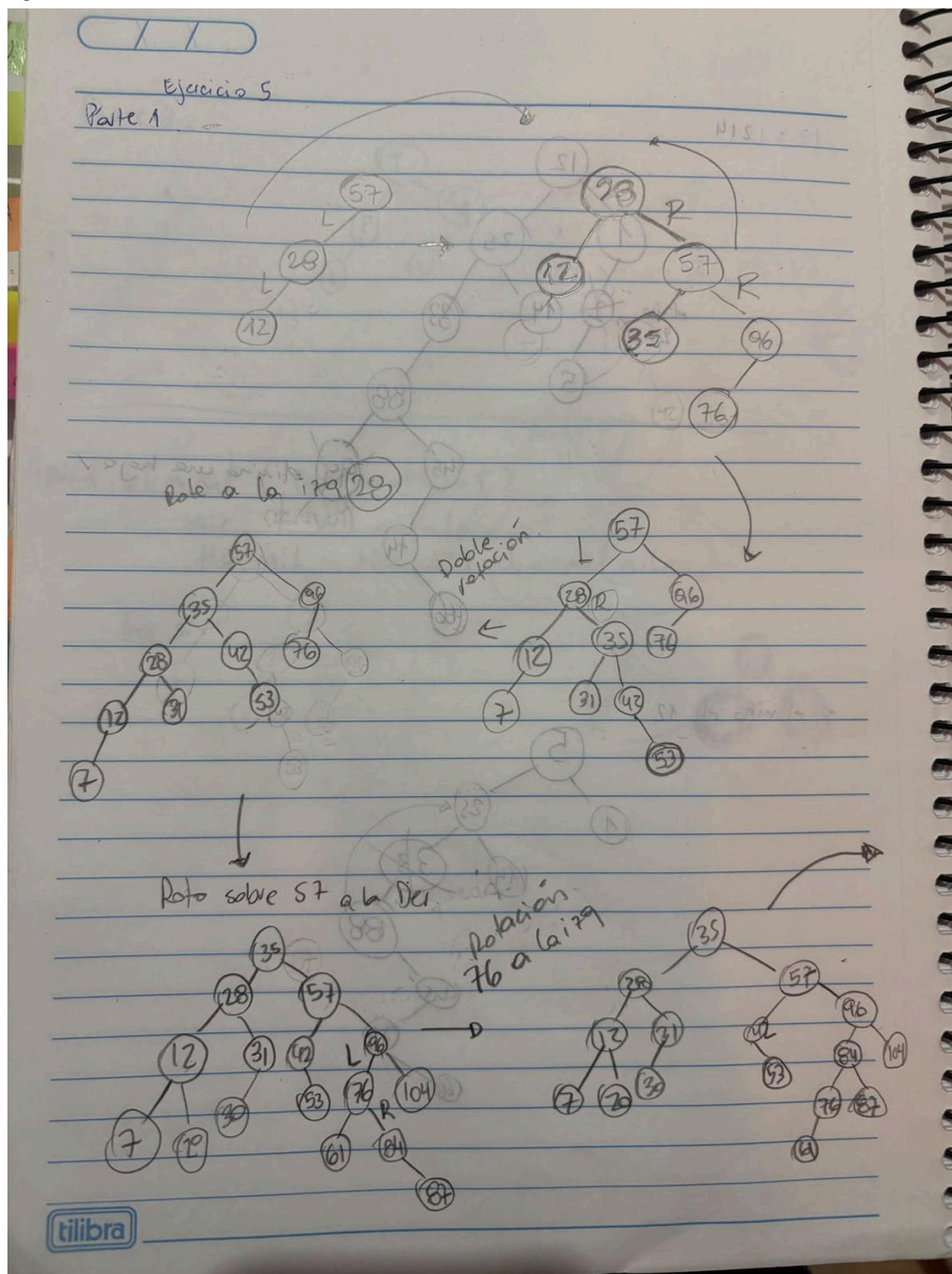
1. Árbol vacío $\rightarrow$ tamaño 0.
2. Árbol de un solo nodo $\rightarrow$ tamaño 1.
3. Insertar 7 claves distintas $\rightarrow$ tamaño 7.

Pseudocódigo:

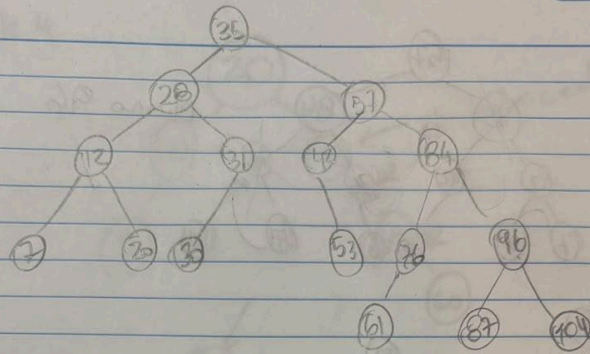
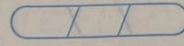
```
function tamaño(nodo)
  si nodo es null entonces
    retornar 0
  fin si
  retornar 1 + tamaño(nodo.hijolzquierdo) + tamaño(nodo.hijoDerecho)
fin function
```

Complejidad: $O(n)$ - misma razón que altura, hay que pasar por cada nodo una sola vez para contarlo

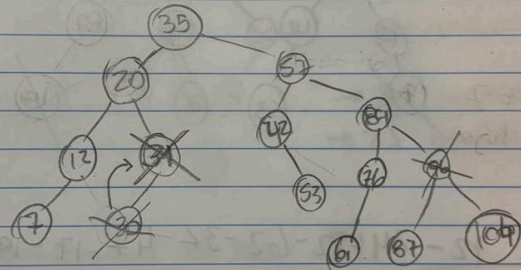
Ejercicio 5:



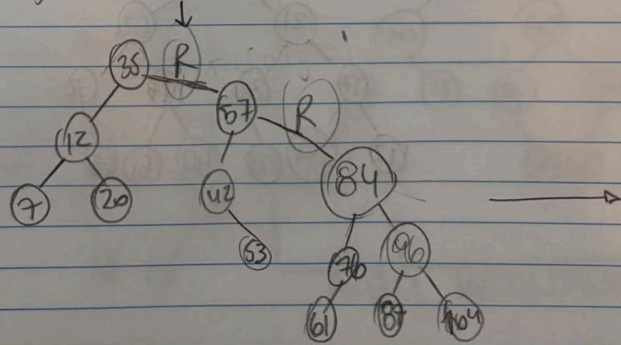
AVL Terminado: PARTE 1

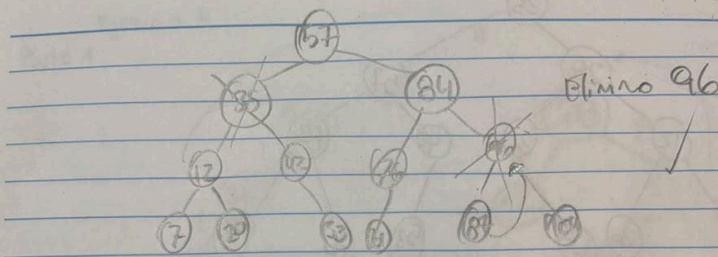


Eliminaciones : (28)

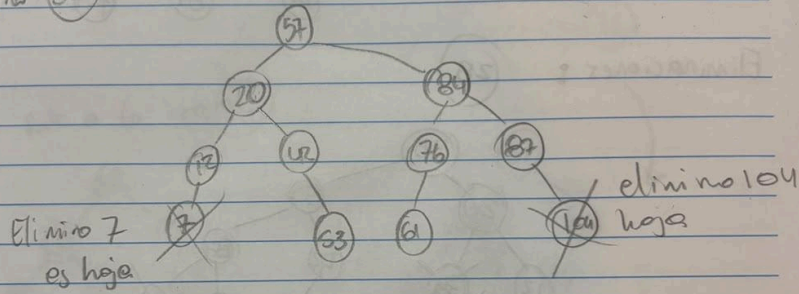


El (31) el más grande de su subArbol i7q es 30 -
El (30) es hoja desaparece de una pero desbalancea el 20

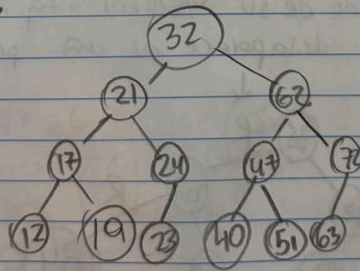


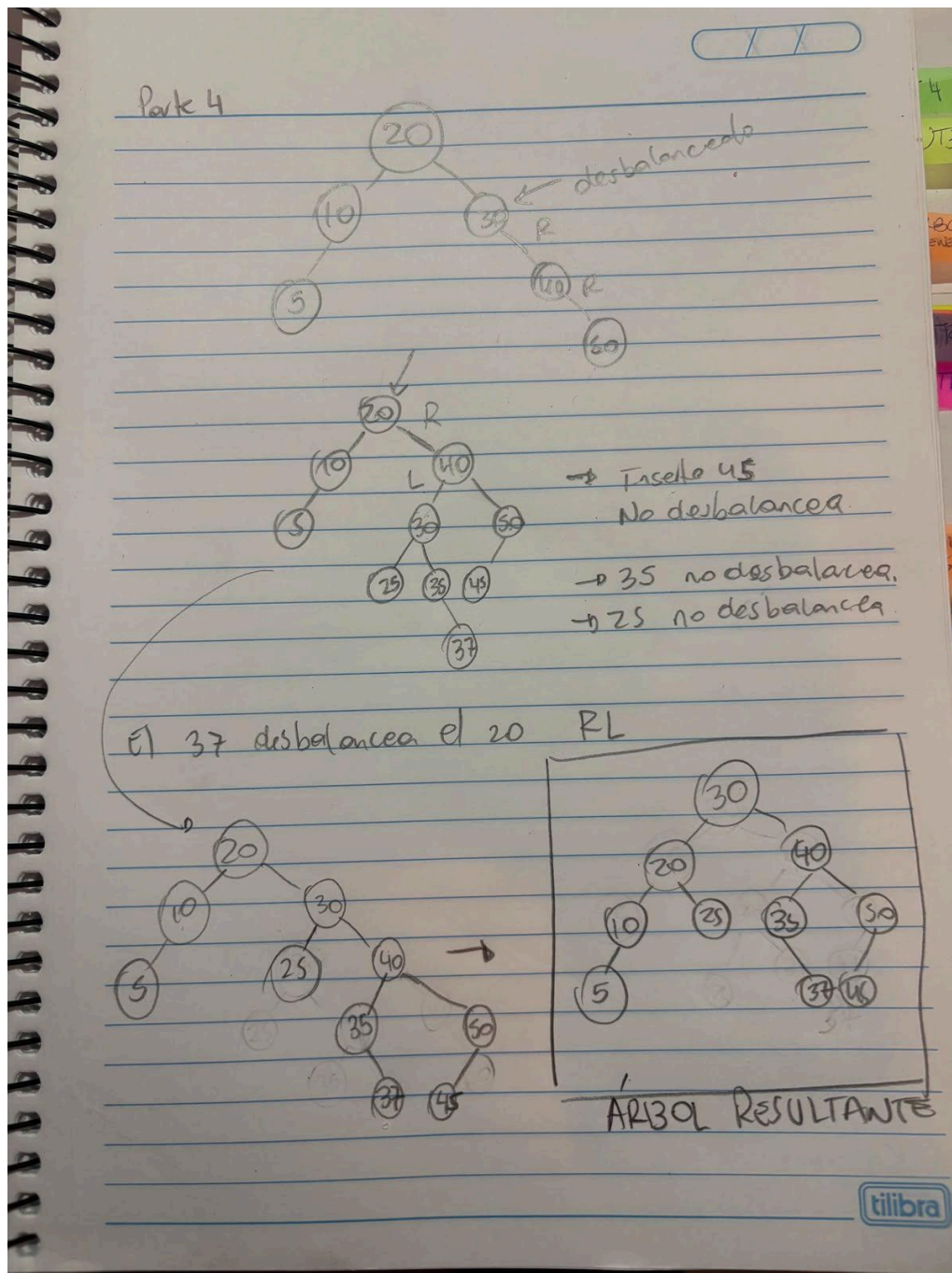


Elimina 35



PARTE 3. 72-24-12-62-32-47-17-19-21-33-40-61
63.





Ejercicio 8:

Parte 1

- 1- (altura 7) es correcta:
- 2- Opción 1 (cinco hojas: C, N, S, V, Z).
- 3-inorden para orden lexicográfico.

MiFunción: devuelve un tipo entero

Comienzo

A <- -1

B <- -1

Si HijoIzquierdo <> nulo entonces

A <- HijoIzquierdo.MiFunción

Fin Si

Si HijoDerecho <> nulo entonces

B <- HijoDerecho.MiFunción

Fin Si

Devolver máximo(A,B) + 1

Fin

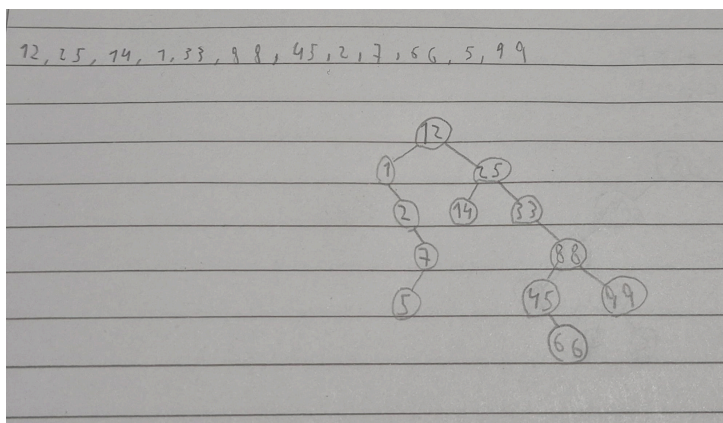
Parte 2

1- la altura del arbol

2-O(n)

Parte 3

1.



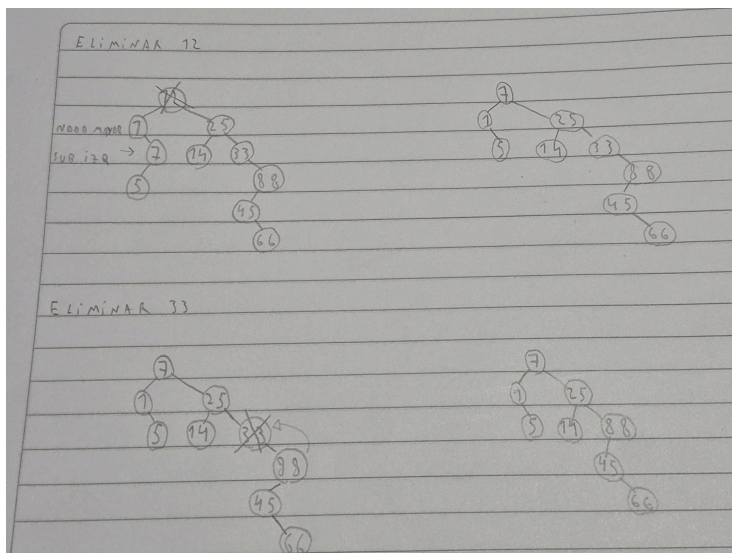
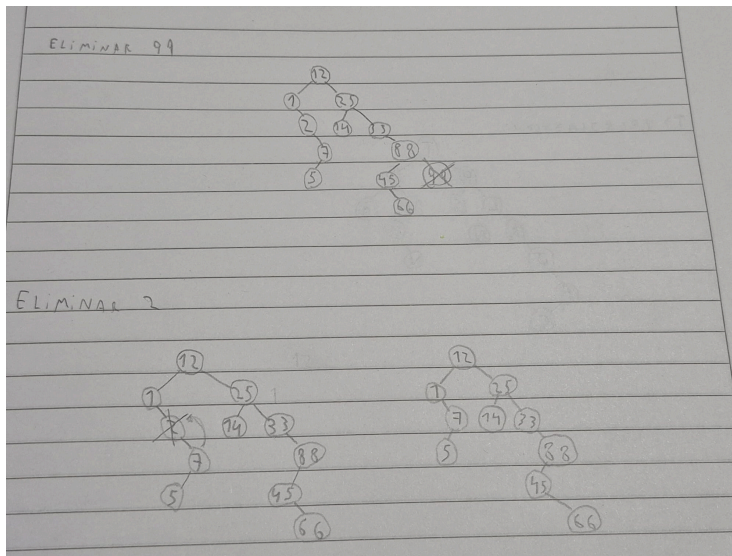
2.

Preorden-12, 1, 2, 7, 5, 25, 14, 33, 88, 45, 66, 99

Inorden-1, 2, 5, 7, 12, 14, 25, 33, 45, 66, 88, 99

Postorden 5, 7, 2, 1, 14, 66, 45, 99, 88, 33, 25, 12

3.



4.

Preorden 7, 1, 5, 25, 14, 88, 45, 66
 Inorden 1, 5, 7, 14, 25, 45, 66, 88
 Postorden 5, 1, 14, 66, 45, 88, 25, 7

Ejercicio 10:

Parte 1

1) Descripción en lenguaje natural:

El método del nodo compara la clave del nuevo nodo contra su propia clave. Si es menor, se fija si tiene hijo izquierdo: si no tiene, ahí mismo se coloca el nuevo nodo; si tiene, le pasa la tarea a ese hijo para que siga bajando. Lo mismo pasa del lado derecho si la clave es mayor. Así se repite hasta encontrar un lugar vacío.

2) Pre y post condiciones:

Pre-condición: el nodo desde el que se invoca no es nulo.

Post-condición: el nuevo nodo queda insertado como hijo izquierdo o derecho de algún nodo del subárbol, respetando la propiedad de BST.

3) Pseudocódigo:

```
metodo insertar(nuevoNodo) en TDANodo
  si nuevoNodo.clave < this.clave entonces
    si this.hijoIzquierdo es null entonces
      this.hijoIzquierdo <- nuevoNodo
    sino
      this.hijoIzquierdo.insertar(nuevoNodo)
  fin si
  sino si nuevoNodo.clave > this.clave entonces
    si this.hijoDerecho es null entonces
      this.hijoDerecho <- nuevoNodo
    sino
      this.hijoDerecho.insertar(nuevoNodo)
  fin si
fin si
fin metodo
```

4) Análisis de complejidad:

$O(h)$, donde h es la altura del árbol. Cada llamada recursiva baja un nivel, comparando y descartando la mitad de las claves restantes en el mejor caso.

Ejercicio 13:

Identificar naves exploradoras:

Descripción en lenguaje natural: Se recorre el árbol completo, visitando cada nodo. Por cada nave visitada se verifica si su clase es "Explorador" si, si se agrega su código a una lista de resultados y se retorna la misma.

Precondición: El árbol existe

Postcondición: Se retorna una lista con los códigos de todas las naves de clase "Explorador" presentes en el árbol.

Pseudocódigo:

```
identificarExploradoras(nodo, lista)
    si nodo es nulo, terminar
    si nodo.clase es "Explorador"
        agregar nodo.codigo a la lista
    identificarExploradoras(hijo izquierdo, lista)
    identificarExploradoras(hijo derecho, lista)
```

Análisis de tiempo de ejecución: Se pasa una sola vez por cada nodo del árbol el tiempo es $O(n)$.

Combustible promedio de naves exploradoras:

Descripción en lenguaje natural: Se recorre todo el árbol, Cada vez que se encuentra una nave "Explorador", se suma su combustible a un total y se cuenta una más.

Precondición: El árbol existe

Postcondición: Se devuelve el promedio de combustible de las naves exploradoras.

Pseudocódigo:

```
acumular(nodo)
  si nodo es nulo, terminar
  si nodo.clase es "Explorador"
    contador = contador + 1
    suma = suma + nodo.combustible
  acumular(hijo izquierdo)
  acumular(hijo derecho)
```

```
promedio()
  contador= 0
  suma =0
  acumular(raiz)
  si contador es 0, devolver 0
  si no, devolver suma / contador
```

Análisis de tiempo de ejecución: Se pasa una sola vez por cada nodo del árbol, el tiempo es $O(n)$.